



African Free Fire Community
africanfreefirecommunity.com

— INTEGRATION GUIDE FOR PARTNER ORGANISATIONS

Sign in with AFC

Partner Integration Guide

AFC is an **OpenID Connect provider**. This guide is everything a partner organisation needs to add "Sign in with AFC" to its site: the exact endpoints, the scope catalogue, the claims each scope produces, the rules that decide what actually reaches you, and a complete worked example.

PROTOCOL	FLOW	VERSION	ISSUED
OpenID Connect	Auth code + PKCE	1.0	3 August 2026

ABOUT THIS DOCUMENT

Audience: developers (and AI coding assistants) integrating "Sign in with AFC" into a partner website or application.

Status of this document: written against the AFC implementation as it stands on 2026-08-03. Every endpoint path, parameter name, claim name, error string and lifetime below was read out of the AFC backend source, not out of the OpenID Connect specification. Where AFC differs from what a spec-reading developer would expect, that difference is called out.

Questions about anything in this guide: info@africanfreefirecommunity.com.

A few items below are noted as not yet decided by AFC. They do not block an integration; they are flagged so you do not build on something provisional.

CONTENTS

1	What this is	4
2	Getting set up: what AFC issues you	5
3	Discovery: bootstrap from one URL	6
	What is in the document	6
	Four things to read carefully in the discovery document	6
4	Endpoint reference (exact paths)	8
5	The authorization code flow with PKCE, step by step	9
	PKCE is REQUIRED. An integration without it will fail.	9
	Step 1: Redirect the player to the authorization endpoint	10
	Step 2: AFC redirects back to your redirect_uri	10
	Step 3: Exchange the code for tokens	10
	Step 4: Verify the ID token	11
	Step 5: Read the userinfo endpoint	12
6	The scope catalogue and the exact claims each one produces	13
	Three places the scope description overpromises	13
	email_verified is meaningful	14
	Claims with no value are absent, not null	14
7	The four gates: why a scope you requested can return nothing	15
	Gate 4, in full	15
	Gates are re-evaluated live, on every call	15
	What this means for your code	16
8	The sub claim is pairwise: the only safe key to store	17
	Do not key on email or username	17
	Operational caveat to be aware of	17

9	Re-consent when you widen your scopes	18
10	Errors and refusals	19
	AFC refusals render a page, they do not redirect	19
	Redirect URL mismatch	19
	Errors that do come back to your callback	19
	Token endpoint errors	20
	UserInfo endpoint errors	20
11	Token lifetimes, refresh and revocation	21
	Refresh tokens rotate. You must store the new one.	21
	Revoking a token from your side	21
	The player can revoke you at any time, without warning	22
	There is no logout endpoint	22
12	Worked end-to-end example (Next.js App Router)	23
	Shared helper	23
	Step 1: start the flow	24
	Step 2: handle the callback	25
	Later: refresh the access token	26
	Using an off-the-shelf library instead	27
13	Common mistakes	28
14	Security expectations of the partner	29
15	Quick reference card	30

1. What this is

AFC (African Free Fire Community) is an **OpenID Connect provider**. Your site is the **relying party**. A player who already has an AFC account clicks "Sign in with AFC" on your site, is sent to AFC to sign in and approve what is shared, and is sent back to you with an authorization code. You exchange that code for an ID token, an access token and a refresh token. From then on you can read that player's AFC data from the userinfo endpoint, without the player signing in again, until they revoke you.

The important difference from "Sign in with Google" is this: **AFC decides, per partner, the maximum set of fields you may ever receive, and the player then consents on top of that.** Google gives every developer the same menu and lets the user pick from it. AFC gives *your organisation* a menu that an AFC admin configured specifically for you, and the player picks from that. On top of both, AFC applies its own protection rules (for example, players under 18 never release contact data). The practical consequence, and the single thing integrations most often get wrong: **you can request a scope you were approved for, the player can approve it, and you can still legitimately receive no claim for it.** Your code must treat every claim other than `sub` as optional. See [section 7](#) (page 15).

AFC implements the standard protocol with no AFC dialect. Any compliant OIDC client library works unmodified. There is no AFC SDK to install.

2. Getting set up: what AFC issues you

There is **no self-serve developer portal and no public app registration**. Dynamic Client Registration (RFC 7591) is disabled on the server. An AFC head admin creates your application record by hand and hands you the credentials.

Who to ask: email info@africanfreefirecommunity.com. AFC issues your credentials and decides which data your application is approved to receive.

You give AFC:

YOU PROVIDE	NOTES
Organisation display name	Shown to the player on the consent screen (" <i>Name</i> wants to use your AFC account").
Logo URL	Optional. Shown on the consent screen. Must be a URL AFC can serve to the player's browser.
Homepage URL	Optional. Shown on the player's Connected Apps page.
One or more redirect URIs	Exact, absolute URIs. See the matching rule below.
The list of data fields you need	AFC turns each one into a per-partner toggle. Every toggle starts OFF.

AFC gives you back:

YOU RECEIVE	NOTES
<code>client_id</code>	A public identifier. Safe to put in a redirect URL.
<code>client_secret</code>	Shown once, at creation time. AFC stores only a hash of it and cannot show it to you again. If you lose it, AFC must issue a new one. Keep it server side only.
The set of scopes you are approved for	Anything outside this set is refused at the authorize endpoint before the player sees anything.

Redirect URI matching is exact. The `redirect_uri` you send on the authorize request must be one of the URIs registered on your application, matched exactly. A trailing slash difference, an added query parameter, `http` where `https` was registered, or a different port all fail. Register every environment you need (production, staging, local development) as separate URIs.

Both `http` and `https` are accepted by the underlying validator, but **register HTTPS redirect URIs for production**. A redirect URI is where an authorization code is delivered, and over plain HTTP that code is exposed in transit.

3. Discovery: bootstrap from one URL

AFC publishes a standard OpenID Connect discovery document. **Read it at runtime and drive your integration from it.** Do not hardcode endpoint paths. A compliant OIDC client library can bootstrap the entire integration from this single URL, and that is the recommended way to integrate.

```
GET https://api.africanfreefirecommunity.com/sso/.well-known/openid-configuration
```

The public base URL is `https://api.africanfreefirecommunity.com`. The SSO surface is mounted at the `/sso/` prefix on the AFC API, NOT on the `africanfreefirecommunity.com` website origin, so every path in this guide hangs off that host. Put it in an environment variable rather than in source, so you can point at a test host if AFC gives you one.

A trailing slash is accepted: `/sso/.well-known/openid-configuration` and `/sso/.well-known/openid-configuration/` both return the document. The response carries `Access-Control-Allow-Origin: *`, so it is readable from a browser.

What is in the document

Concrete example, with the values AFC's configuration actually produces:

```
{
  "issuer": "https://api.africanfreefirecommunity.com/sso",
  "authorization_endpoint": "https://api.africanfreefirecommunity.com/sso/authorize/",
  "token_endpoint": "https://api.africanfreefirecommunity.com/sso/token/",
  "userinfo_endpoint": "https://api.africanfreefirecommunity.com/sso/userinfo/",
  "jwks_uri": "https://api.africanfreefirecommunity.com/sso/.well-known/jwks.json",
  "scopes_supported": [
    "openid", "profile", "email",
    "afc.freefire", "afc.team", "afc.history",
    "afc.stats", "afc.ranking", "afc.standing"
  ],
  "response_types_supported": [
    "code", "token", "id_token", "id_token token",
    "code token", "code id_token", "code id_token token"
  ],
  "subject_types_supported": ["pairwise"],
  "id_token_signing_alg_values_supported": ["RS256", "HS256"],
  "token_endpoint_auth_methods_supported": ["client_secret_post", "client_secret_basic"],
  "code_challenge_methods_supported": ["plain", "S256"],
  "claims_supported": ["sub"],
  "prompt_values_supported": ["none", "login"],
  "client_id_metadata_document_supported": false
}
```

JSON

Note the `issuer` value has **no trailing slash** and **includes the `/sso` path segment**. Use it verbatim when validating the `iss` claim of an ID token.

Four things to read carefully in the discovery document

Three of these are values a partner could reasonably misread, and the first is a confirmation rather than a warning. The discovery document is generated by the underlying library, so some of its values describe what the library CAN do rather than what AFC will accept from you.

1. `subject_types_supported` says `["pairwise"]`, and it means it. The same player gets a different `sub` at every partner. Build for pairwise. See [section 8 \(page 17\)](#).
2. `claims_supported` lists only `["sub"]`. This is not the claim catalogue. AFC computes claims per request, so the library cannot enumerate them at discovery time. The real catalogue is [section 6 \(page 13\)](#) of this document.
3. `response_types_supported` lists seven values, including implicit and hybrid types. In practice partner applications are created with the authorization code grant, so `response_type=code` is the only one that will work for you. Do not use `token`, `id_token`, or any hybrid combination.
4. `code_challenge_methods_supported` includes `"plain"`. Do not use `plain`. Use `S256`. PKCE with `plain` provides no protection against an attacker who can observe the authorization request.

There is **no** `end_session_endpoint`. RP-initiated logout is not enabled on AFC. Signing a player out of your site does not sign them out of AFC, and there is no AFC endpoint to ask for that. See [section 11 \(page 21\)](#).

4. Endpoint reference (exact paths)

All paths are relative to the AFC API base URL. Every path below was read from AFC's URL configuration, not assumed from library defaults. **Note the trailing slashes: they are part of the path.**

PURPOSE	METHOD	PATH	AUTH
Discovery document	GET	<code>/sso/.well-known/openid-configuration</code>	none
JSON Web Key Set	GET	<code>/sso/.well-known/jwks.json</code>	none
Authorization	GET	<code>/sso/authorize/</code>	player's AFC browser session
Token	POST	<code>/sso/token/</code>	client credentials
Userinfo	GET or POST	<code>/sso/userinfo/</code>	<code>Authorization: Bearer <access_token></code>
Token revocation	POST	<code>/sso/revoke_token/</code>	client credentials

Notes on each:

- `/sso/authorize/` is AFC's own view, not the library's. AFC mounts its consent screen at this path ahead of the library's route, so it takes precedence. This is where the four refusal checks in [section 10 \(page 19\)](#) run.
- The revocation path is `/sso/revoke_token/`, not `/sso/revoke/`. This trips people up. It follows RFC 7009 semantics.
- `/sso/introspect/` is registered by the underlying library but is **not part of the partner contract**. Do not build against it. It is not part of the partner contract today. Ask info@africanfreefirecommunity.com if you have a case for it.
- `/sso/logout/` is registered but **disabled**. RP-initiated logout is off in AFC's configuration, so this endpoint is not usable and no `end_session_endpoint` appears in discovery.
- `/sso/register/` (Dynamic Client Registration) is **disabled**. Applications are created by an AFC admin only.
- `/sso/me/connected-apps/` exists but is **not for partners**. It is AFC's own player-facing API behind the Connected Apps page in the player's AFC profile, and it authenticates with an AFC session token, not with your OAuth tokens.

5. The authorization code flow with PKCE, step by step

PKCE is REQUIRED. An integration without it will fail.

AFC sets `PKCE_REQUIRED: True`. This is not advisory and it is not only for public clients. **Even though you are a confidential client with a `client_secret`, you must send `code_challenge` and `code_challenge_method` on the authorization request and `code_verifier` on the token request.** If you omit them, the authorization request is rejected or the token exchange fails. This is the single most common cause of a first integration attempt failing.

Generating the verifier and challenge:

The `code_verifier` is a high-entropy random string, 43 to 128 characters, from the URL-safe alphabet. The `code_challenge` is the base64url encoding (no padding) of the SHA-256 digest of the verifier.

Node.js:

```
import crypto from "node:crypto";

const base64url = (buf) =>
  buf.toString("base64").replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");

const codeVerifier = base64url(crypto.randomBytes(32)); // 43 chars
const codeChallenge = base64url(
  crypto.createHash("sha256").update(codeVerifier).digest()
); // 43 chars
```

JAVASCRIPT

Python:

```
import base64, hashlib, secrets

code_verifier = base64.urlsafe_b64encode(secrets.token_bytes(32)).decode().rstrip("=")
code_challenge = base64.urlsafe_b64encode(
  hashlib.sha256(code_verifier.encode()).digest()
).decode().rstrip("=")
```

PYTHON

Shell (for manual testing):

```
CODE_VERIFIER=$(openssl rand -base64 32 | tr '+/' '-_' | tr -d '=')
CODE_CHALLENGE=$(printf '%s' "$CODE_VERIFIER" | openssl dgst -binary -sha256 | openssl base64 | tr '+/' '-_' | tr -d '=')
```

SHELL

Store the verifier server side, bound to this one browser session, in an httpOnly cookie or a server-side session. You need it again at step 4, and only the browser that started the flow may complete it.

Step 1: Redirect the player to the authorization endpoint

```
GET https://api.africanfreefirecommunity.com/sso/authorize/
?response_type=code
&client_id=<your client_id>
&redirect_uri=https%3A%2F%2Fpartner.example%2Fauth%2Fafc%2Fcallback
&scope=openid%20profile%20afc.freefire
&state=<random, unguessable, stored in the session>
&nonce=<random, unguessable, stored in the session>
&code_challenge=<the S256 challenge>
&code_challenge_method=S256
```

PARAMETER	REQUIRED	VALUE
<code>response_type</code>	yes	<code>code</code> . Nothing else works for a partner application.
<code>client_id</code>	yes	The identifier AFC issued you.
<code>redirect_uri</code>	yes	Must match a registered URI exactly, and must be URL-encoded in the query string.
<code>scope</code>	yes	Space-separated. Must include <code>openid</code> . Everything else is from section 6 (page 13).
<code>state</code>	yes in practice	CSRF defence. Random, stored in the session, compared on the callback.
<code>nonce</code>	strongly recommended	Random, stored in the session, compared against the <code>nonce</code> claim in the ID token. Replay defence.
<code>code_challenge</code>	yes	The base64url SHA-256 of your verifier.
<code>code_challenge_method</code>	yes	<code>S256</code> .
<code>approval_prompt</code>	no	Send <code>auto</code> to ask AFC to reuse an existing approval and skip the consent screen. AFC's default is to show the screen every time. See section 9 (page 18) for when <code>auto</code> is ignored.

What the player sees. If they are not signed in to AFC, AFC bounces them to the AFC website login carrying the full authorization request, then returns them here. That extra hop is normal and invisible to you. Once signed in, they get AFC's consent screen: your organisation's name and logo, a plain-language list of exactly what you will be able to see, and Allow / Deny buttons. The screen renders in the player's own language (English, French or Portuguese).

Step 2: AFC redirects back to your `redirect_uri`

On approval:

```
https://partner.example/auth/afc/callback?code=abc123...&state=<the state you sent>
```

On denial, or on a protocol-level error that AFC considers safe to redirect (see [section 10](#) (page 19) for those it does not):

```
https://partner.example/auth/afc/callback?error=access_denied&state=<the state you sent>
```

Before doing anything else, compare the returned `state` against the value you stored. If they differ, or the stored value is missing, abort. Do not exchange the code.

Step 3: Exchange the code for tokens

The authorization code is **single use** and **expires in 60 seconds**. Exchange it immediately, server side.

```
curl -X POST https://api.africanfreefirecommunity.com/sso/token/ \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "grant_type=authorization_code" \
-d "code=abc123..." \
-d "redirect_uri=https://partner.example/auth/afc/callback" \
-d "client_id=YOUR_CLIENT_ID" \
-d "client_secret=YOUR_CLIENT_SECRET" \
-d "code_verifier=$CODE_VERIFIER"
```

SHELL

`redirect_uri` must be **byte-identical** to the one you sent at step 1. `code_verifier` is the raw verifier, not the challenge.

Client authentication: AFC accepts `client_secret_post` (the form fields shown above) and `client_secret_basic` (HTTP Basic with `client_id` as the username and `client_secret` as the password). Either is fine. Pick one.

Response, 200:

```
{
  "access_token": "kR2h...",
  "expires_in": 36000,
  "token_type": "Bearer",
  "scope": "openid profile afc.freefire",
  "refresh_token": "9Xz4...",
  "id_token": "eyJhbGciOiJIUzI1NiIsImtpZCI6..."
}
```

JSON

Note the `scope` field: it is the scope actually granted. It may be narrower than what you asked for. Read it rather than assuming.

Response, 400, for a replayed code, an expired code, a wrong verifier, a mismatched `redirect_uri`, or bad client credentials. The body is the standard OAuth 2.0 error object, for example `{"error": "invalid_grant"}`.

Step 4: Verify the ID token

The ID token is a JWT signed with **RS256**, using a key published at the JWKS endpoint. Verify it. Do not decode it without verifying.

Check all of:

CHECK	EXPECTED VALUE
Signature	Valid against a key from <code>jwks_uri</code> , matched by the <code>kid</code> in the JWT header.
<code>iss</code>	Exactly the <code>issuer</code> from discovery, for example <code>https://api.africanfreefirecommunity.com/sso</code> .
<code>aud</code>	Your <code>client_id</code> .
<code>exp</code>	In the future.
<code>nonce</code>	Equal to the nonce you generated at step 1.

Decoded payload (claims beyond the standard set depend on your scopes and the four gates):

```
{
  "iss": "https://api.africanfreefirecommunity.com/sso",
  "aud": "YOUR_CLIENT_ID",
  "sub": "3f7a1c9e0b5d4a28f61c8e0d7b3a5f92c4e18d60a7b2f34c9d1e5a08b6c7f2d3",
  "exp": 1786000000,
  "iat": 1785964000,
  "auth_time": 1785964000,
  "nonce": "the nonce you sent",
  "at_hash": "...",
  "preferred_username": "PlayerName",
  "country": "NG",
  "locale": "en",
  "ff_uid": "8390224792"
}
```

JSON

AFC puts the same claims in the ID token as at the userinfo endpoint. This is deliberate: the two are computed by one function so they cannot disagree. You may read the profile data straight out of the verified ID token on first login and skip the userinfo call.

■ Step 5: Read the userinfo endpoint

Use this later, whenever you want fresh data. The claims are recomputed on every call, so a field AFC has since switched off disappears here even though your access token is still valid.

```
curl https://api.africanfreefirecommunity.com/sso/userinfo/ \
-H "Authorization: Bearer kR2h..."
```

SHELL

Response, 200:

```
{
  "sub": "3f7a1c9e0b5d4a28f61c8e0d7b3a5f92c4e18d60a7b2f34c9d1e5a08b6c7f2d3",
  "preferred_username": "PlayerName",
  "country": "NG",
  "locale": "en",
  "ff_uid": "8390224792"
}
```

JSON

GET and **POST** both work. An expired, revoked or unknown access token returns **401** with a **WWW-Authenticate: Bearer** header. The exact error body is produced by the underlying OAuth library and is not something AFC customises, so **branch on the status code, not on the body text**.

6. The scope catalogue and the exact claims each one produces

These are all nine scopes AFC defines. There are no others. Request only what you need: every extra scope is another thing the player can refuse, and a wider request forces a fresh consent screen.

The claim names are not what a reader would guess. AFC uses `ff_uid`, `afc_team`, `afc_history`, `afc_stats`, `afc_ranking` and `afc_standing` (underscores), while the scope names use dots (`afc.freefire`, `afc.team`, and so on). Copy the claim names from this table exactly.

SCOPE	WHAT THE PLAYER IS TOLD THEY ARE SHARING	CLAIMS PRODUCED	SHAPE OF THE VALUE
<code>openid</code>	"Confirm who you are on AFC"	<code>sub</code>	String. 64-character lowercase hex (a SHA-256 digest). Pairwise per partner. Always present.
<code>profile</code>	"Your in-game name, avatar, country and language"	<code>preferred_username</code> , <code>picture</code> , <code>country</code> , <code>locale</code>	<code>preferred_username</code> : string, the AFC username. <code>country</code> : free-text string, commonly a two-letter code such as "NG" but not schema-enforced , so treat it as opaque. <code>locale</code> : string, one of "en", "fr", "pt". <code>country</code> and <code>locale</code> are omitted when the player has not set them.
<code>email</code>	"Your email address"	<code>email</code> , <code>email_verified</code>	<code>email</code> : string. <code>email_verified</code> : boolean, and it is real : it reflects whether the player completed AFC's emailed activation code.
<code>afc.freefire</code>	"Your Free Fire UID"	<code>ff_uid</code>	String, the player's Free Fire UID. Omitted when the player has no UID on file.
<code>afc.team</code>	"Your current team and your role in it"	<code>afc_team</code>	Object: <code>{"name": "Team Name", "role": "captain"}</code> . <code>role</code> falls back to "member" when no management role is set. The whole claim is omitted when the player is not on a team.
<code>afc.history</code>	"Tournaments you have played"	<code>afc_history</code>	Array of objects, newest registration first, capped at 50 entries : <code>[{"event": "Event Name", "slug": "event-slug"}]</code> . Empty array when the player has no history. No placement or result data is included.
<code>afc.stats</code>	"Your match statistics"	<code>afc_stats</code>	Object: <code>{"matches": 128, "kills": 940}</code> . Both integers, 0 when the player has no recorded matches. Only these two figures.
<code>afc.rankings</code>	"Your AFC rank and ranking points"	<code>afc_ranking</code>	Object for the most recent scored month: <code>{"points": 1420, "rank": 17, "month": "2026-07-01"}</code> . <code>month</code> is an ISO date string. The whole claim is omitted when the player has never been scored. No tier is included (tier is a team attribute on AFC, not a player one).
<code>afc.standing</code>	"Whether your AFC account is in good standing"	<code>afc_standing</code>	Object: <code>{"in_good_standing": true}</code> . <code>false</code> when the player is banned or their account is not active.

Three places the scope description overpromises

The consent text is what the player reads; the claim table above is what your code actually receives. Where they differ, the table wins:

- `profile` releases an avatar as the standard `picture` claim (added 2026-08-03). It is an absolute URL on AFC's API domain. It is **omitted** for players who have not uploaded one, so render a fallback.
- `afc.history` releases event name and slug only. No placement, no result, no date.
- `afc.stats` releases match count and kill count only. No damage, assists, or per-event breakdown.

■ `email_verified` is meaningful

`email_verified` is derived from the player's account being active, which on AFC means they completed the emailed activation code at signup (accounts created through Google are active immediately, because Google already verified the address). It is safe to gate account linking on it. It was previously hardcoded to `true`; that was corrected on 2026-08-03.

■ Claims with no value are absent, not null

AFC strips every claim whose value would be `null` before sending the payload. So an AFC player with no team produces a response with **no** `afc_team` **key at all**, rather than `"afc_team": null`. Write your parsing accordingly: check for key presence, and never assume `response.afc_team.name` is reachable.

7. The four gates: why a scope you requested can return nothing

This is the part of AFC's model that surprises integrators, so it is stated bluntly. **A field reaches you only if all four of these agree. If any one refuses, the claim is simply absent from the payload. It is not `null`, and it is not an error.**

GATE	CONTROLLED BY	MEANING
1. AFC's per-partner toggle	An AFC admin	Is this field switched on for <i>your</i> organisation? Every toggle defaults OFF.
2. The requested scope	You	Did you actually ask for it in this authorization request?
3. Player consent	The player	Did this player press Allow for that scope, and is the grant still live?
4. AFC's own rules	AFC policy, evaluated live	The rules below.

Gate 4, in full

These are applied on **every** ID token issuance and **every** userinfo call, against the player's state at that moment.

- 1. Under-18 players never release contact data.** If the player's date of birth puts them under 18, the `email` scope is dropped, whatever your toggle says and whatever the player consented to. **This fails closed:** a player with no date of birth on file, or an unparseable one, is treated as a minor and gets no `email` either. Expect a meaningful share of your players to have no `email` claim.
- 2. A player with statistics hidden releases no statistics.** AFC players have a `stats_visible` setting. When it is off, the `afc.stats` scope is dropped and no `afc_stats` claim appears. This too fails closed: a player for whom the flag is unset is treated as hidden.
- 3. A suspended account releases only its standing.** If the AFC account is not active, the effective scope set collapses to `openid` plus `afc.standing`. You still get `sub`, and you still get `afc_standing` (which will report `in_good_standing: false`) if you were approved for it. **Everything else disappears**, including profile and email, even though the token itself is still valid. This is deliberate: whether a player is in good standing is the one question a partner has a legitimate reason to ask about a suspended account.

Note the interaction with the authorize endpoint: a player whose account is already suspended cannot start a new sign-in at all (they get a refusal, see [section 10 \(page 19\)](#)). Gate 4 rule 3 is what happens to a player who is suspended **after** you already hold their tokens.

Gates are re-evaluated live, on every call

There is no caching. If AFC switches off your `afc.freefire` toggle this afternoon, the `ff_uid` claim vanishes from your **next userinfo call** with a token issued this morning. You do not get an error, a webhook, or a token invalidation. The field is just no longer there.

What this means for your code

JAVASCRIPT

```
// WRONG. Throws for a player with no team, or one whose scope was gated.
const teamName = userinfo.afc_team.name;

// WRONG. "" is a plausible value for country; this silently mislabels it.
const country = userinfo.country || "unknown";

// RIGHT.
const teamName = userinfo.afc_team?.name ?? null;
const hasTeam = "afc_team" in userinfo;
const country = "country" in userinfo ? userinfo.country : null;
```

Design your data model so that every AFC field except `sub` is nullable, and so that a field disappearing on a later refresh is a normal event rather than a data-integrity failure. Do not, for example, wipe a stored team name just because one userinfo call omitted it: decide explicitly whether absence means "no longer applies" or "no longer shared".

8. The `sub` claim is pairwise: the only safe key to store

`sub` is the primary key for an AFC player on your side. Store it. Key on it. Never key on anything else.

AFC issues **pairwise** subject identifiers. The `sub` for one player is derived from AFC's server secret, your application's internal record and the player's internal record, and it is a 64-character SHA-256 hex digest. The consequences:

- **The same player has a different `sub` at every partner.** Two partner sites cannot compare their `sub` values to work out that they are looking at the same person. That is the point.
- `sub` is **opaque**. It encodes nothing you can decode, and you cannot compute it yourself.
- `sub` is **stable for you** across every sign-in, token refresh and userinfo call, for as long as the player's AFC account and your application record both exist.

Store it as a string of exactly 64 characters and treat it as an opaque token.

■ Do not key on email or username

Both change, and both will break your account linking:

- **Email** changes when a player updates their AFC account, and is **absent entirely** for every player under 18 and for every player whose date of birth AFC does not have. You cannot rely on receiving it at all.
- **Username** (`preferred_username`) is an in-game name that AFC players do change.

If you already have local accounts and want to link them to AFC accounts by email on first sign-in, that is a reasonable one-time bootstrap. But persist the `sub` at that moment and match on `sub` from then on.

■ Operational caveat to be aware of

AFC's `sub` derivation is tied to a server-side secret. AFC has documented internally that rotating that secret would change every partner's view of every player and break all existing account links. AFC is aware of this and it is not expected to happen, but if you ever see every returning player arrive as a new `sub`, that is the cause and you should contact AFC rather than merging accounts. If you ever see this, contact info@africanfreefirecommunity.com immediately rather than merging accounts.

9. Re-consent when you widen your scopes

If you request a scope the player has not already approved for you, AFC shows the consent screen again. You cannot suppress this.

The rule as implemented:

- AFC compares the scopes on this request against the scopes on the player's **live, unexpired tokens** for your application. Expired tokens do not count as evidence of consent.
- If the request asks for **anything** not in that set, AFC forces the consent screen.
- **Sending `approval_prompt=auto` does not suppress it.** AFC overrides your `auto` with `force` in exactly this case. This is enforced in AFC's own code rather than left to the library, specifically so that a configuration change cannot relax it.
- Narrowing does not re-prompt. Asking for fewer scopes than the player already approved is fine.
- By default, AFC shows the consent screen **on every authorization**, whether or not anything widened. Sending `approval_prompt=auto` is what asks AFC to reuse a live approval, and AFC honours that only when nothing widened.

Two practical consequences:

1. **You cannot silently gain a scope.** If AFC switches on a new toggle for you, the first time you actually request that scope the player is asked again. Plan the user-facing moment: a player mid-task being bounced to an AFC consent screen needs to understand why.
2. **Do not request scopes speculatively.** Requesting the full catalogue "in case we need it later" means a longer consent list, more refusals, and more absent claims.

There is also no per-partner bypass. AFC pins off the underlying library's "skip the consent screen" flag on every save, because every partner application is a third party by definition.

10. Errors and refusals

AFC refusals render a page, they do not redirect

Four checks run at `/sso/authorize/` **before** anything is rendered and before any code is issued. When one fails, AFC returns **HTTP 400 with a plain-text message rendered on AFC's own page**. It does **not** redirect to your `redirect_uri` with an `error` parameter.

This is deliberate, and it is a security property, not a bug. Bouncing a refusal to whatever `redirect_uri` sits in the query string would make AFC an open redirector: an attacker could craft a link that fails on purpose and lands the player on a phishing page carrying an AFC referrer. Nothing in a refusal path echoes attacker-controlled input into a `Location` header. AFC has an explicit regression test asserting that no refusal ever returns a 302.

What this means for you: a refused authorization never comes back to your callback. The player is left on an AFC error page. You will see the flow start and never complete. Do not build a "the callback always fires" assumption into your session handling; time out pending flows.

The four refusals, with the exact English messages:

CONDITION	HTTP	MESSAGE
The <code>client_id</code> matches no application	400	Unknown application.
Your application's status is suspended	400	This application is suspended.
The player's AFC account is not active	400	Your AFC account cannot sign in to partner sites.
You requested a scope your organisation is not approved for	400	This application requested data it is not approved for.

Do not string-match these messages. They are translated into the player's language, so a French player sees the French text. They are shown to a human, not parsed by a machine. If you need to detect these conditions, detect the absence of a callback.

The scope refusal is worth understanding: AFC checks your requested scopes against your approved set **at the front door**, before the consent screen. The claim-building layer would strip an over-broad scope anyway, but letting the request through would show the player a consent screen listing data your organisation is not permitted to receive. So a request for even one unapproved scope fails the whole authorization. Keep your requested scope list in sync with what AFC approved for you.

Redirect URI mismatch

A `redirect_uri` that does not match a registered URI is rejected by the underlying library, and it also does not redirect (verified by test). The player sees an error page at AFC. Same practical consequence: no callback fires.

Errors that do come back to your callback

Once the request is valid and the consent screen has rendered, ordinary OAuth errors return to your `redirect_uri` as query parameters. The one you will actually see is the player pressing Deny:

```
https://partner.example/auth/afc/callback?error=access_denied&state=<your state>
```

Always handle `error` on the callback before looking for `code`.

Token endpoint errors

Standard OAuth 2.0 JSON errors with HTTP 400, for example:

```
{"error": "invalid_grant"}
```

`JSON`

Common causes: the code was already used (codes are single use and AFC tests this explicitly), the code expired (60 seconds), the `code_verifier` does not match the `code_challenge`, the `redirect_uri` differs from step 1, or the client credentials are wrong.

Userinfo endpoint errors

401 for an expired, revoked or unknown access token, with a `WWW-Authenticate: Bearer` header. Branch on the status code. On 401, refresh the access token and retry once; if the refresh also fails, treat the player as disconnected and prompt them to sign in with AFC again.

11. Token lifetimes, refresh and revocation

These are the values in AFC's configuration as it stands. AFC does not override the library defaults for any of them, so they are the library defaults. Read `expires_in` from the token response rather than hardcoding these, in case AFC tunes them.

ITEM	LIFETIME	NOTES
Authorization code	60 seconds	Single use. Exchange it immediately.
Access token	36000 seconds (10 hours)	Returned as <code>expires_in</code> .
ID token	36000 seconds (10 hours)	Same as the access token.
Refresh token	No expiry	It remains valid until it is used (and rotated), revoked by you, revoked by the player, or deleted when AFC suspends your application.

Refresh tokens rotate. You must store the new one.

AFC has refresh token rotation enabled, with a **zero-second grace period**. Every successful refresh returns a new `refresh_token` and invalidates the one you sent, immediately, with no overlap window.

```
curl -X POST https://api.africanfreefirecommunity.com/sso/token/ \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "grant_type=refresh_token" \
-d "refresh_token=9Xz4..." \
-d "client_id=YOUR_CLIENT_ID" \
-d "client_secret=YOUR_CLIENT_SECRET"
```

SHELL

The response has the same shape as the authorization code exchange, including a fresh `refresh_token`.

Implications you must design for:

- **Persist the new refresh token in the same transaction that consumes the old one.** If your process crashes between the HTTP response and the database write, that player's connection is dead and they must sign in again.
- **Serialise refreshes per player.** Two concurrent refreshes with the same token means one succeeds and the other gets a 400, and depending on ordering you can lose the valid token. Use a lock or a single-flight pattern.
- There is no grace period, so "retry the same refresh token on a network timeout" is not safe. If a refresh times out, re-read your stored token before retrying.

Note that `scope` is not a parameter you send on refresh: the new tokens carry the scopes the previous ones did. To change scopes, run a fresh authorization (which will re-prompt if you widened, see [section 9](#) (page 18)).

Revoking a token from your side

RFC 7009, at `/sso/revoke_token/`:

```
curl -X POST https://api.africanfreefirecommunity.com/sso/revoke_token/ \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "token=9Xz4..." \
-d "client_id=YOUR_CLIENT_ID" \
-d "client_secret=YOUR_CLIENT_SECRET"
```

SHELL

Revoking the refresh token kills the ability to mint new access tokens (AFC has a test asserting a subsequent refresh returns 400). Do this when a user deletes their account on your side or unlinks AFC, as a matter of hygiene.

■ The player can revoke you at any time, without warning

Every AFC player has a Connected Apps page in their AFC profile listing the partner organisations they have signed in to, what each one can see, and a Remove button. Pressing Remove deletes, for your application and that player: all access tokens, all refresh tokens, any unexchanged authorization codes, and all issued ID tokens.

You get no notification. The first you know of it is a 401 from userinfo and a 400 from refresh. Handle that as a normal state transition (mark the AFC connection as disconnected, invite the player to reconnect), not as an outage.

The same is true if AFC suspends your application: authorization requests start returning `This application is suspended.`

Not available yet. AFC's data model has a `deletion_webhook_url` field on partner applications, intended for a standards-based deletion signal (OpenID RISC) when a player revokes. **It is not implemented or fired today**, and it is documented internally as a later piece of work. Do not build against it. Ask info@africanfreefirecommunity.com about the timeline if you need push notification of revocation. Until then, treat a token that stops working as a revocation.

■ There is no logout endpoint

RP-initiated logout is disabled. Signing a player out of your site has no effect on their AFC session, and there is no AFC endpoint that will end it. If you want a "sign out of AFC too" affordance, the only honest thing you can offer is a link to AFC. AFC has not committed to enabling RP-initiated logout.

12. Worked end-to-end example (Next.js App Router)

A complete, minimal reference integration. Node runtime, App Router, TypeScript. It generates PKCE, redirects, handles the callback, verifies the ID token against JWKS, and reads userinfo.

Dependency: `jose` for JWT verification.

```
pnpm add jose
```

SHELL

Environment variables (`.env.local`, never committed):

```
AFC_ISSUER=https://api.africanfreefirecommunity.com/sso
AFC_CLIENT_ID=your-client-id
AFC_CLIENT_SECRET=your-client-secret
AFC_REDIRECT_URI=https://partner.example/api/auth/afc/callback
```

Shared helper

```
// lib/afc.ts
// Discovery lookup and JWKS handling for "Sign in with AFC".
// Endpoints are READ FROM DISCOVERY, never hardcoded, so AFC can move them.
import { createRemoteJWKSet } from "jose";

export const AFC_ISSUER = process.env.AFC_ISSUER!;
export const AFC_CLIENT_ID = process.env.AFC_CLIENT_ID!;
export const AFC_CLIENT_SECRET = process.env.AFC_CLIENT_SECRET!;
export const AFC_REDIRECT_URI = process.env.AFC_REDIRECT_URI!;

// Request only what you need. Every extra scope is one more thing the player can
// refuse, and widening this list later forces a fresh consent screen.
export const AFC_SCOPES = "openid profile afc.freefire";

type Discovery = {
  issuer: string;
  authorization_endpoint: string;
  token_endpoint: string;
  userinfo_endpoint: string;
  jwks_uri: string;
};

let cached: { doc: Discovery; at: number } | null = null;

export async function discovery(): Promise<Discovery> {
  // One hour is plenty. The document is stable; refetching per request is wasteful.
  if (cached && Date.now() - cached.at < 3_600_000) return cached.doc;
  const res = await fetch(`${AFC_ISSUER}/.well-known/openid-configuration`);
  if (!res.ok) throw new Error(`AFC discovery failed: ${res.status}`);
  const doc = (await res.json()) as Discovery;
  cached = { doc, at: Date.now() };
  return doc;
}

// Module-level so the key set is fetched once and cached by jose, not per request.
let jwks: ReturnType<typeof createRemoteJWKSet> | null = null;
```

TYPESCRIPT

CONTINUES ON THE NEXT PAGE

CONTINUED

```
export async function afcJwks() {
  if (!jwks) jwks = createRemoteJWKSet(new URL((await discovery()).jwks_uri));
  return jwks;
}
```

Step 1: start the flow

TYPESCRIPT

```
// app/api/auth/afc/login/route.ts
// Sends the player to AFC. Generates the PKCE pair, state and nonce, and stashes all
// three in httpOnly cookies so only this browser can complete the flow.
import crypto from "node:crypto";
import { NextResponse } from "next/server";
import {
  AFC_CLIENT_ID, AFC_REDIRECT_URI, AFC_SCOPES, discovery,
} from "@lib/afc";

const base64url = (buf: Buffer) =>
  buf.toString("base64").replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");

export async function GET() {
  // PKCE is REQUIRED by AFC. Omitting these two parameters fails the flow.
  const codeVerifier = base64url(crypto.randomBytes(32));
  const codeChallenge = base64url(
    crypto.createHash("sha256").update(codeVerifier).digest()
  );
  const state = base64url(crypto.randomBytes(16));
  const nonce = base64url(crypto.randomBytes(16));

  const { authorization_endpoint } = await discovery();
  const url = new URL(authorization_endpoint);
  url.searchParams.set("response_type", "code");
  url.searchParams.set("client_id", AFC_CLIENT_ID);
  url.searchParams.set("redirect_uri", AFC_REDIRECT_URI);
  url.searchParams.set("scope", AFC_SCOPES);
  url.searchParams.set("state", state);
  url.searchParams.set("nonce", nonce);
  url.searchParams.set("code_challenge", codeChallenge);
  url.searchParams.set("code_challenge_method", "S256");

  const res = NextResponse.redirect(url.toString());
  const opts = {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax" as const,
    path: "/",
    maxAge: 600, // the flow should not take ten minutes
  };
  res.cookies.set("afc_verifier", codeVerifier, opts);
  res.cookies.set("afc_state", state, opts);
  res.cookies.set("afc_nonce", nonce, opts);
  return res;
}
```


Step 2: handle the callback

TYPESCRIPT

```
// app/api/auth/afc/callback/route.ts
// Validates state, exchanges the code (60-second window), verifies the ID token
// against AFC's JWKS, then reads userinfo. `sub` is the only field keyed on.
import { NextRequest, NextResponse } from "next/server";
import { jwtVerify } from "jose";
import {
  AFC_CLIENT_ID, AFC_CLIENT_SECRET, AFC_REDIRECT_URI, afcJwks, discovery,
} from "@lib/afc";

export async function GET(req: NextRequest) {
  const params = req.nextUrl.searchParams;

  // 1. The player pressed Deny, or an OAuth-level error came back. Handle this FIRST.
  const error = params.get("error");
  if (error) {
    return NextResponse.redirect(new URL(`/login?afc_error=${error}`, req.url));
  }

  // 2. CSRF: the state must match what we issued, in this browser.
  const code = params.get("code");
  const state = params.get("state");
  const expectedState = req.cookies.get("afc_state")?.value;
  const verifier = req.cookies.get("afc_verifier")?.value;
  const nonce = req.cookies.get("afc_nonce")?.value;
  if (!code || !state || !expectedState || state !== expectedState || !verifier) {
    return NextResponse.json({ error: "invalid_state" }, { status: 400 });
  }

  const doc = await discovery();

  // 3. Exchange the code. `redirect_uri` must be byte-identical to step 1, and
  // `code_verifier` is the RAW verifier, not the challenge.
  const tokenRes = await fetch(doc.token_endpoint, {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body: new URLSearchParams({
      grant_type: "authorization_code",
      code,
      redirect_uri: AFC_REDIRECT_URI,
      client_id: AFC_CLIENT_ID,
      client_secret: AFC_CLIENT_SECRET,
      code_verifier: verifier,
    }),
  });
  if (!tokenRes.ok) {
    return NextResponse.json(
      { error: "token_exchange_failed", detail: await tokenRes.text() },
      { status: 400 }
    );
  }
  const tokens = await tokenRes.json() as {
    access_token: string;
    refresh_token: string;
    id_token: string;
    expires_in: number;
    scope: string;
  };
}
```

CONTINUES ON THE NEXT PAGE

CONTINUED

```
};

// 4. Verify the ID token: signature, issuer, audience, expiry, then the nonce.
const { payload } = await jwtVerify(tokens.id_token, await afcJwks(), {
  issuer: doc.issuer,          // "https://.../sso", no trailing slash
  audience: AFC_CLIENT_ID,
});
if (payload.nonce !== nonce) {
  return NextResponse.json({ error: "nonce_mismatch" }, { status: 400 });
}

// 5. Read userinfo. Optional here (the ID token carries the same claims), but this
//    is the call you repeat later to get fresh data.
const infoRes = await fetch(doc.userinfo_endpoint, {
  headers: { Authorization: `Bearer ${tokens.access_token}` },
});
if (!infoRes.ok) {
  return NextResponse.json({ error: "userinfo_failed" }, { status: 502 });
}
const info = await infoRes.json();

// 6. EVERY field except `sub` is optional. A missing claim is normal: AFC's toggle
//    may be off, the player may not have consented, or an AFC rule may have gated it
//    (under-18 players release no email, hidden stats release nothing, and so on).
const player = {
  afcSub: info.sub as string,          // 64-char hex. The ONLY join key.
  username: (info.preferred_username as string) ?? null,
  email: (info.email as string) ?? null,          // often absent, by design
  country: (info.country as string) ?? null,
  locale: (info.locale as string) ?? null,
  freeFireUid: (info.ff_uid as string) ?? null,
  teamName: (info.afc_team as { name?: string } | undefined)?.name ?? null,
};

// await upsertUserByAfcSub(player, tokens); // key on afcSub, never on email
// Store tokens.refresh_token; it ROTATES on every refresh, so persist the new one.

const res = NextResponse.redirect(new URL("/", req.url));
for (const c of ["afc_verifier", "afc_state", "afc_nonce"]) res.cookies.delete(c);
return res;
}
```

■ Later: refresh the access token

TYPESCRIPT

```
// lib/afc-refresh.ts
// AFC rotates refresh tokens with NO grace period: the token you send is dead the
// instant this succeeds. Persist the new one before doing anything else, and hold a
// per-player lock so two concurrent refreshes cannot race and lose the live token.
import { AFC_CLIENT_ID, AFC_CLIENT_SECRET, discovery } from "@lib/afc";

export async function refreshAfcTokens(storedRefreshToken: string) {
  const doc = await discovery();
  const res = await fetch(doc.token_endpoint, {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body: new URLSearchParams({
```

CONTINUES ON THE NEXT PAGE

CONTINUED

```
    grant_type: "refresh_token",
    refresh_token: storedRefreshToken,
    client_id: AFC_CLIENT_ID,
    client_secret: AFC_CLIENT_SECRET,
  }},
});

// 400 here usually means the player revoked you from their AFC Connected Apps page,
// or AFC suspended the application. Mark the connection disconnected; do not retry.
if (!res.ok) throw new Error(`afc_refresh_failed:${res.status}`);

return res.json() as Promise<{
  access_token: string;
  refresh_token: string;    // NEW. Save it.
  id_token: string;
  expires_in: number;
  scope: string;
}>;
}
```

■ Using an off-the-shelf library instead

The hand-rolled version above is written out in full so nothing is ambiguous, but a compliant OIDC client library pointed at the discovery URL is less code and less to get wrong. Whatever you use, confirm it sends PKCE (`code_challenge` / `code_verifier`) and verifies the ID token signature against `jwks_uri` . Configure it with:

- Discovery URL: `<AFC base>/sso/.well-known/openid-configuration`
- Client authentication: `client_secret_post` or `client_secret_basic`
- Response type: `code`
- PKCE method: `S256`
- Scopes: `openid` plus whatever AFC approved for you

13. Common mistakes

- 1. Forgetting PKCE.** AFC requires it for confidential clients too. If your library treats PKCE as a public-client-only feature, turn it on explicitly. This is the number one cause of a first integration failing.
- 2. Keying on email or username.** Email changes, and is absent entirely for under-18 players and for any player whose date of birth AFC does not hold. Username is an in-game name that players change. **Key on `sub`.**
- 3. Assuming a requested scope always returns a value.** All four gates must agree. Treat every claim except `sub` as optional, on every call, including calls that succeeded yesterday.
- 4. Assuming a missing claim is `null`.** AFC omits the key entirely. `userinfo.afc_team.name` throws; `userinfo.afc_team?.name` does not.
- 5. Hardcoding endpoints instead of reading discovery.** Also: hardcoding `/sso/revoke/` when the real path is `/sso/revoke_token/`, and dropping the trailing slashes, which are part of every AFC path.
- 6. Mismatching the redirect URI.** It must match a registered URI exactly at step 1, and be byte-identical again in the token exchange at step 3. A trailing slash, an added query parameter or a different scheme all fail.
- 7. Relying on `claims_supported` to enumerate what you will receive.** It lists only `sub`, a library limitation rather than a description of AFC. The real catalogue is [section 6 \(page 13\)](#). (`subject_types_supported` IS accurate: pairwise.)
- 8. Losing the rotated refresh token.** Every refresh invalidates the token you sent, with no grace period. Persist the new one atomically, and do not refresh concurrently for the same player.
- 9. Assuming every claim is present.** Any field can be absent for a legitimate reason (see section 7). Code defensively rather than indexing straight into the payload.
- 10. Expecting a refusal to come back to your callback.** AFC's four refusals render a 400 page at AFC and never redirect. Your callback simply never fires, so time out pending flows.
- 11. Sitting on a 60-second authorization code.** Exchange it in the callback handler, not on a queue.
- 12. Expecting a logout endpoint.** There is none.

14. Security expectations of the partner

AFC is handing you data about real players, some of them minors. These are not optional.

- 1. Keep `client_secret` server side.** Never in client-side JavaScript, a mobile app binary, a public repository, or an environment variable exposed to the browser (in Next.js, never prefix it with `NEXT_PUBLIC_`). It is shown once and AFC cannot re-display it. Rotate it through AFC if it is ever exposed.
- 2. Run the token exchange server side.** The authorization code and `client_secret` must never touch the browser.
- 3. Validate `state` on every callback.** Generate it randomly, store it bound to the browser session, compare it, and reject on mismatch. This is your CSRF defence.
- 4. Use `nonce` and check it.** Generate it randomly, send it on the authorization request, and compare it against the `nonce` claim in the verified ID token. This is your replay defence.
- 5. Verify the ID token signature against JWKS.** Fetch keys from the `jwks_uri` in discovery, match on the `kid` in the JWT header, and verify `iss`, `aud` and `exp` as well as the signature. Never trust a decoded-but-unverified JWT. Cache the key set, and let your library re-fetch on an unknown `kid` so AFC can rotate signing keys.
- 6. HTTPS everywhere.** Redirect URIs, your callback, and every AFC call. No exceptions in production.
- 7. Store PKCE verifier, state and nonce in httpOnly, `Secure`, `SameSite=Lax` cookies or a server-side session,** with a short lifetime, and delete them once the flow completes.
- 8. Store tokens encrypted at rest and never log them.** Access tokens, refresh tokens and ID tokens are credentials. Keep them out of application logs, error trackers and analytics.
- 9. Honour revocation promptly.** A 401 from userinfo or a 400 from refresh means the player or AFC cut you off. Stop using the tokens and delete them.
- 10. Do not re-share AFC data.** What AFC releases to you is scoped to your organisation and consented to by the player for your organisation. Passing it to a third party is outside what the player agreed to.
- 11. Request the minimum scopes you need.** Fewer scopes means a shorter consent list, a higher approval rate and less to protect.
- 12. Handle minors' data carefully.** AFC deliberately withholds contact data for under-18 players. Design your product so that an account with no email address is fully functional rather than blocked.

15. Quick reference card

Base URL	https://api.africanfreefirecommunity.com	
Issuer	https://api.africanfreefirecommunity.com/sso (no trailing slash)	
Discovery	GET /sso/.well-known/openid-configuration	
JWKS	GET /sso/.well-known/jwks.json	
Authorize	GET /sso/authorize/	
Token	POST /sso/token/	
Userinfo	GET /sso/userinfo/	Authorization: Bearer <access_token>
Revoke	POST /sso/revoke_token/	
Flow	authorization code + PKCE (S256). PKCE IS MANDATORY.	
Response type	code	
Client auth	client_secret_post client_secret_basic	
ID token alg	RS256	
Scopes	openid profile email afc.freefire afc.team afc.history afc.stats afc.ranking afc.standing	
Claims	sub (64-hex, PAIRWISE, the only safe key) preferred_username, country, locale <- profile email, email_verified <- email ff_uid <- afc.freefire afc_team {name, role} <- afc.team afc_history [{event, slug}] max 50 <- afc.history afc_stats {matches, kills} <- afc.stats afc_ranking {points, rank, month} <- afc.ranking afc_standing {in_good_standing} <- afc.standing Missing claims are ABSENT, not null. All four gates must agree.	
Lifetimes	auth code	60 s, single use
	access token	36000 s (10 h)
	id token	36000 s (10 h)
	refresh token	no expiry, ROTATES on every use, no grace period
Not available	RP-initiated logout, dynamic client registration, revocation webhooks, self-serve app registration	

Questions, credential issues, scope changes and application suspensions:

info@africanfreefirecommunity.com